



cpi'fp

Los Enlaces

DESARROLLO WEB EN ENTORNO SERVIDOR

2º DESARROLLO DE APLICACIONES WEB

SESIONES, HELPERS Y EJEMPLO CRUD

1. SESIONES EN LARAVEL

Como es lógico, Laravel también proporciona su propio sistema de manejo de **variables de sesión**, es decir, variables persistentes en el servidor asociadas a cada cliente.

Las variables de sesión de Laravel son mucho más seguras y poderosas que las variables de sesión estándar de PHP.

Las sesiones se configuran en `/config/sessions.php`. Existen varios **drivers** de sesión: **files** (driver por defecto), **memcached y redis** (para aplicaciones en producción) y **database** (para seguridad adicional).

1. SESIONES EN LARAVEL

Laravel maneja dos tipos de variable según su persistencia:

1. **Variables flash:**

Son variables de sesión que solo duran una petición y luego se *autodestruyen*. Se usan típicamente para enviar un *feedback* o mensaje de retroalimentación al usuario.

Imagina el típico formulario de *login*. En caso de producirse un error, lo habitual es que la aplicación nos muestre de nuevo ese formulario con un mensaje del tipo “Usuario no reconocido”.

1. SESIONES EN LARAVEL

Para eso, haríamos lo siguiente en el controlador. Observa el uso del método *with()* para crear una variable flash de sesión llamada *mensaje*:

```
return ('login/form')->with('mensaje', 'Usuario no reconocido');
```

En la vista, podemos acceder a esa variable flash. Por ejemplo, así:

```
@if (session('mensaje'))  
    {{ session('mensaje'); }}  
@endif // A partir de este momento, la variable flash se destruye y cualquier  
intento de acceder a ella provocará un error de ejecución
```

1. SESIONES EN LARAVEL

2. Variables convencionales:

Las variables de sesión convencionales se manejan con la clase *Session*, que tiene un montón de métodos estáticos para crear variables, destruirlas, consultarlas, etc. Los métodos más útiles son:

- **put()** → Almacena una variable de sesión:

```
Session::put('nombre-variable', 'valor');
```

- **push()** → Elimina una variable de sesión:

```
Session::push('nombre-variable');
```

1. SESIONES EN LARAVEL

- **get()** → Devuelve el valor de una variable de sesión:

```
$v = Session::get('nombre-variable', 'valor-por-defecto');
```

- **all()** → Devuelve todas las variables de sesión en un array:

```
$a = Session::all('nombre-variable', 'valor');
```

- **flush()** → Elimina todas las variables de sesión:

```
$a = Session::flush();
```

- **flash()** → Crea manualmente una variable de sesión de tipo *flash*:

```
$a = Session::flash('nombre-variable', 'valor');
```

2. LARAVEL BREEZE

La **autenticación** de usuarios, es decir, el sistema de *login* seguido de la creación de una o varias variables de sesión asociadas al usuario que se acaba de loguear, es un componente habitual de muchas aplicaciones web.

Hasta la versión 5, Laravel proporcionaba un sistema de autenticación integrado en su código pero a partir de Laravel 6, los desarrolladores decidieron aligerar el núcleo de Laravel lo máximo posible, y sacaron del sistema muchos componentes, incluyendo el sistema de autenticación.



2. LARAVEL BREEZE

Actualmente, Laravel proporciona los denominados ***Starter Kits***, que son componentes que se pueden instalar mediante *composer* para realizar esas tareas que se extrajeron del núcleo de Laravel.

Para la autenticación, Laravel dispone de dos *Starter Kits*, llamados ***Breeze*** y ***Jetstream***. Vamos a ver el primero, que es más sencillo pero suficiente en la mayor parte de las situaciones.

2. LARAVEL BREEZE

Laravel Breeze contiene todo el código necesario para crear un sistema de autenticación completo y seguro capaz, entre otras cosas, de:

- Hacer el Login e iniciar la sesión
- Registrar nuevos usuarios
- Recuperar contraseñas olvidadas
- Confirmar el registro de usuarios mediante email

2. LARAVEL BREEZE

Para instalar *Laravel Breeze*, debes abrir un terminal en tu servidor web y ejecutar estos comandos:

```
$ composer require laravel/breeze --dev  
$ php artisan breeze:install  
$ npm install  
$ npm run dev  
$ php artisan migrate
```

Los comandos *npm* sirven para compilar el CSS y el JS de *Breeze*. El último actualiza las migraciones para crear sus tablas adicionales.

2. LARAVEL BREEZE

Una vez hecho esto, *Breeze* creará automáticamente varias rutas en un enrutador especial, `/routes/auth.php`. Entre ellas:

```
Routes::get("/login")           // Para mostrar el formulario de login
Routes::post("/login")          // Para procesar el formulario de login
Routes::post("/logout")         // Para cerrar la sesión
Routes::get("/register")        // Para mostrar el formulario de registro
Routes::post("/register")       // Para procesar el formulario de registro
```

Puedes probarlas escribiéndolas en tu navegador. Comprobarás que funcionan correctamente.

2. LARAVEL BREEZE

También se crearán varios controladores, como *LoginController* y *RegisterController*, en *App/Http/Controllers/Auth*. Y, varias vistas, como *auth/login.blade.php*, *register.blade.php* y *layouts/app.blade.php*.

Por último, se crea una vista *home* de ejemplo (*dashboard.blade.php*) a la que llegamos después de hacer login. Dicha vista la puedes cambiar en */app/providers/RouteServiceProvider.php* y redirigirla a la que te interese.

¡Y listo! Ya tienes hecho un sistema de login completo y superseguro. Solo te queda adaptar esas vistas y controladores a tus necesidades.



3. AUTENTICACIÓN, VISTAS Y MIDDLEWARES

En las vistas, tenemos un par de directivas de Blade muy útiles relacionadas con las sesiones: **@auth** y **@guest**.

```
@auth
...    // Este código solo se ejecuta si existe un usuario logueado
@endauth
@guest
...    // Este código solo se ejecuta si NO existe usuario logueado
@endguest
```

3. AUTENTICACIÓN, VISTAS Y MIDDLEWARES

Además, podemos acceder a los datos del usuario mediante la clase Auth:

```
$user = Auth::user() // Devuelve el usuario actualmente logueado o null si no  
hay ninguna sesión abierta.  
if (Auth::check()) { // Devuelve true si el usuario actual está logueado.  
    ...  
}
```

Puedes consultar más métodos de Auth en la [documentación oficial](#).

3. AUTENTICACIÓN, VISTAS Y MIDDLEWARES

Los **middlewares** de Laravel (*App/Http/Middleware*) son componentes que filtran las peticiones HTTP que llegan a la aplicación. Literalmente, se ponen *en medio* de cualquier petición al servidor, y de ahí su nombre.

Hay dos *middlewares* importantes relacionados con la autenticación en Laravel: *Authenticate* (que tiene un alias: ***auth***) y *RedirectIfAuthenticated* (alias ***guest***). Los alias se definen en *App/Http/Kernel.php*.

El *middleware* *auth* puede usarse en el enrutador:

```
Route::get('/ruta-a-proteger', 'Controlador@metodo')->middleware('auth');
```



3. AUTENTICACIÓN, VISTAS Y MIDDLEWARES

También podemos usar estos *middlewares* en el constructor de nuestros controladores para protegerlos en todo o en parte.

```
public function __construct() {  
    // Solo usuarios logueados podrán acceder a cualquier función de este controlador:  
    $this->middleware("auth");  
    // Solo usuarios logueados podrán acceder a los métodos create() y edit():  
    $this->middleware("auth")->only("create", "edit");  
    // Solo usuarios logueados podrán acceder al controlador excepto a show():  
    $this->middleware("auth")->except("show");  
}
```


4. HELPERS

Un ***helper*** es un componente del framework diseñado para **facilitar alguna tarea** típica en el desarrollo de una aplicación web.

Por ejemplo: el *helper url()* sirve para generar una ruta absoluta a partir de una relativa, de modo que la ruta absoluta siempre funcione, sea cual sea el servidor donde ejecutes la aplicación.

```
<a href="{{ url('/users/login') }}">Volver</a>
```

Cuando actúe, traducirá la expresión anterior por este código HTML:

```
<a href='https://miservidor.com/users/login'>Volver</a>
```

4. HELPERS

Eso permite que la ruta sea correcta aunque muevas la aplicación de un servidor a otro, sin necesidad de modificar el código fuente.

El uso de los *helpers* es optativo, solo son ayudantes, y el programador/a debe decidir si le resultan útiles o no.

Los *helpers* cambian mucho de una versión a otra de Laravel, por lo que es recomendable que eches un vistazo a la documentación oficial para saber qué *helpers* están disponibles en tu versión de Laravel. Puedes encontrar una lista completa en <https://laravel.com/docs/10.x/helpers>.

4. HELPERS

ROUTE HELPER:

Es parecido a *url()*, pero sirve para **rutas con nombre**. Por ejemplo, si en el enrutador tenemos una ruta con nombre como ésta:

```
Route::get("mi-ruta", "mi-controlador@metodo")->name("nombre-ruta");
```

...podemos referirnos a ella en cualquier vista como:

```
<a href="{ url('mi-ruta') }">Texto</a>           <!--Primera forma-->  
<a href="{ route('nombre-ruta') }">Texto</a>     <!--Segunda forma-->
```

La segunda forma es mejor que la primera, porque nos permite cambiar en cualquier momento la dirección que ve el usuario sin modificar el código fuente del resto de la aplicación.

4. HELPERS

REQUEST HELPER:

Este *helper* **proporciona información sobre la petición** (GET, POST o la que sea) con la que se cargó la página.

El *helper request()* devuelve un objeto con varios métodos para acceder a esa información. Estos son los más útiles:

- **request()->url()** → Devuelve un string con la ruta actual (completa).
- **request()->path()** → Devuelve un string con la ruta actual (solo desde la raíz de la aplicación, sin http ni el nombre del servidor).

4. HELPERS

REQUEST HELPER:

- **request()->is("ruta")** → Devuelve true si coincide con la ruta actual.
- **request()->input("campo")** → Devuelve el valor de "campo" (enviado desde formulario).
- **request()->all()** → Devuelve un array con todos los campos.
- **request()->has("campo")** → Devuelve true si en la petición existe un campo con el nombre indicado.
- **request()->isMethod("método")** → Devuelve true si la petición se hizo por el método indicado (POST, GET, PUT, etc).

4. HELPERS

REQUEST HELPER:

El *helper request()* puede usarse en las vistas (como `request()->url()`, por ejemplo) o inyectarse en las funciones del controlador como un argumento. En este último caso, podemos acceder a todos los datos enviados mediante un formulario a través del objeto `$r`. Por ejemplo:

```
public function mi_función(Request $r) {  
    $name = $r->name;  
    $email = $r->email;  
    $pass = $r->password;    ...etc...    }
```

4. HELPERS

REDIRECT HELPER:

Muy útil cuando queremos **redirigir al usuario hacia otra URL o acción** (por ejemplo, para evitar que al pulsar F5 se reenvíen los datos de un formulario).

Admite varias formas:

```
return redirect('user/login');  
return redirect()->action('LoginController@login');  
return back();
```

4. HELPERS

AUTH HELPER:

Como hemos visto en la sección de sesiones y autenticación, este helper permite **saber si existe algún usuario autenticado** en la aplicación. **auth()->user()** nos devolverá el usuario autenticado (como un objeto de tipo **User**) o *null* si nadie ha hecho login. Con él podemos acceder a todos los datos del usuario. Por ejemplo:

```
$user = auth()->user();  
Bienvenido/a, {{ $user->name }}.  
Este es su email: {{ $user->email }}
```


4. HELPERS

ERRORS HELPER:

Se utiliza para **conocer y mostrar los errores de validación de un formulario**. El objeto **\$errors** está disponible en todas las vistas gracias a que un *Middleware* (*ShareErrorsFromSession*) la inyecta automáticamente.

Algunos métodos útiles de este objeto son:

- **\$errors->all()** → devuelve un array con todos los errores detectados.
- **\$errors->any()** → devuelve true si se ha detectado algún error.
- **\$errors->first("campo")** → devuelve el primer error de todos los que puedan afectar al campo indicado.

5. FLUJO DE TRABAJO CON LARAVEL

Pasos (los cuatro primeros son solo para aplicaciones nuevas):

- Instalar y configurar la nueva aplicación.
- Crear los modelos (se supone que ya tendrás la BD diseñada).
- Crear las migraciones y los *seeders*.
- Lanzar las migraciones y *seeders* para crear y poblar la BD.
- Crear en el enrutador las entradas de la funcionalidad que vas a programar.
- Crear el controlador (si no existe) para la funcionalidad que vas a programar.
- Crear las funciones del controlador necesarias.
- Crear las funciones del modelo necesarias (si no existen ya).
- Crear las vistas necesarias.
- Probar.
- Repetir los pasos 5-10 para cada funcionalidad adicional.

6. CRUD (Create-Read-Update-Delete)

Vamos a desarrollar una pequeña **aplicación web desde cero**. Se trata de un fragmento de otra aplicación más grande: una tienda online o tal vez un sistema de gestión de almacén. Nosotros vamos a desarrollar la parte de mantenimiento de productos.

Para ello, supondremos que en la base de datos existe una **tabla** llamada ***products*** con los campos *id*, *name*, *description* y *price*.

Vamos a construir el controlador, el modelo y las vistas necesarias para hacer el **CRUD** completo de esta tabla con Laravel, sin olvidarnos de las migraciones, los seeders y, por supuesto, el enrutador.

6. CRUD (Create-Read-Update-Delete)

Creamos el proyecto *prueba* desde terminal situándonos en el directorio */laravel* y ejecutando:

```
PS C:\xampp\htdocs\laravel> laravel new prueba
```

En phpMyAdmin creamos la base de datos *laravel* y en el archivo *.env* de nuestro proyecto configuramos la conexión a la base de datos:

```
DB_CONNECTION=mysql  
DB_HOST=localhost  
DB_PORT=3306
```

```
DB_DATABASE=laravel  
DB_USERNAME=root  
DB_PASSWORD=
```

6. CRUD (Create-Read-Update-Delete)

Migraciones:

Para esta pequeña aplicación solo necesitamos una migración, puesto que solo tenemos que crear una tabla.

La migración se crea con el comando `php artisan make:migration create_products_table`, la cual se escribe en el archivo `/database/migrations/[timestamp]_create_products_table.php` con el siguiente contenido:

https://drive.google.com/file/d/1uzlbZZ3yLy-U4HbFNbD7qfWn5u7_UhBE/view?usp=drive_link



6. CRUD (Create-Read-Update-Delete)

Migraciones:

A este archivo le hemos añadido las siguientes líneas para crear los campos *name*, *description* y *price*.

```
Schema::create('products', function (Blueprint $table) {  
    $table->id();  
    $table->string('name', 100);  
    $table->string('description', 500);  
    $table->double('price', 5, 2);  
    $table->timestamps();  
});
```

6. CRUD (Create-Read-Update-Delete)

Seeders:

En este *seeder* vamos a cargar unos cuantos datos de prueba. Obviamente, puedes cambiarlos por los que tú quieras.

El seeder se crea con el comando `php artisan make:seeder ProductTableSeeder`, que generará el archivo `/database/seeder/ProductTableSeeder.php`.

https://drive.google.com/file/d/103o0zF6XKmQvm9FdoZceODT1E4ctgFSu/view?usp=drive_link

6. CRUD (Create-Read-Update-Delete)

Seeders:

Recuerda que, para poder lanzar el seeder automáticamente con `php artisan migrate:fresh --seed` u otro comando similar, tienes que editar el archivo *DatabaseSeeder.php* y añadir la línea `$this->call(ProductTableSeeder::class);` al método *run()*.

En cualquier caso, siempre puedes lanzar el *seeder* manualmente en cualquier momento con `php artisan db:seed --class=ProductTableSeeder`.

6. CRUD (Create-Read-Update-Delete)

Enrutador:

El enrutador de la aplicación está en `/routes/web.php`. Basta con abrirlo y añadir esta línea:

```
Route::resource('product', 'ProductController');
```

Alternativamente, podrías crear a mano las siete entradas correspondientes a las siete rutas de un servidor REST. El resultado sería el mismo, pero si defines manualmente las rutas, tienes más control sobre cómo son exactamente.



6. CRUD (Create-Read-Update-Delete)

Enrutador:

O podrías hacer algún cambio más profundo a nivel técnico. Por ejemplo, que la petición para hacer *delete* llegue por GET en lugar de por DELETE (así no tendrías que usar un botón de formulario para lanzar el borrado de un producto y podrías lanzarlo con un link).

Eso sí: ten en cuenta que, si haces algún cambio de este tipo en las rutas, tu servidor ya no será 100% REST.



6. CRUD (Create-Read-Update-Delete)

Enrutador:

Hay una posibilidad intermedia: respetar las 7 rutas estándar REST y *añadir* alguna adicional que te venga bien, como el borrado mediante

```
// Estas son las 7 rutas estándar REST:  
Route::resource('products', 'ProductController');  
  
// Añadimos una ruta NO ESTÁNDAR para borrar productos mediante GET  
Route::get('product/delete/{product}', 'ProductController@destroy')-  
>name('product.myDestroy');
```



6. CRUD (Create-Read-Update-Delete)

Controlador:

El controlador de productos se crea con el comando `php artisan make:controller ProductController`.

El archivo se generará en `/app/Http/Controllers/ProductController.php`. Tendrás que rellenar el código de los 7 métodos REST con algo como esto:

https://drive.google.com/file/d/162EEad1F4nh7o98mvYH-psGAVVCLRaVT/view?usp=drive_link



6. CRUD (Create-Read-Update-Delete)

Modelo:

El modelo se crea con el comando `php artisan make:model Product`.

El archivo con el modelo se generará en *app/models/product.php*.

No es necesario que toques este archivo: puedes dejarlo, de momento, tal y como lo ha generado Artisan.

6. CRUD (Create-Read-Update-Delete)

Vistas (plantilla principal):

La plantilla principal o *master layout* debes crearla en *resources/views/master.blade.php*.

Por supuesto, puedes hacerla como quieras, pero puedes usar de momento el *master layout* que vimos en el tema sobre Blade. Luego ya lo modificarás a tu gusto.

https://drive.google.com/file/d/11xKkAUCwHYCTK87JigjPYMPEPQynnpAh/view?usp=drive_link

6. CRUD (Create-Read-Update-Delete)

Vistas (todos los productos):

La vista con todos los productos la llamaremos *all.blade.php* y puede tener un aspecto como este:

https://drive.google.com/file/d/1bPvvizhGibgwJfynS1GnThbvRjhliT9O/view?usp=drive_link

Vistas (creación/modificación de productos):

El archivo de la vista será *form.blade.php* y su contenido puede ser algo así:

https://drive.google.com/file/d/178IYPMh-QeuGAlq6wL2BHdw-tHBmL_uB/view?usp=drive_link

6. CRUD (Create-Read-Update-Delete)

Funcionamiento:

Con la sintaxis utilizada para esta aplicación también hay que modificar el archivo `/app/Providers/RouteServiceProvider.php`:

```
Route::prefix('api')
    ->middleware('api')
    ->namespace('App\Http\Controllers')    <----- AÑADE ESTO
Route::middleware('web')
    ->namespace($this->namespace)
    ->namespace('App\Http\Controllers')    <----- AÑADE ESTO
```


6. CRUD (Create-Read-Update-Delete)

Funcionamiento:

Ahora ha llegado el momento de comprobar si tu aplicación funciona.

Primero, lanza las migraciones y los seeders con `php artisan migrate:fresh --seed`. Asegúrate de haber añadido tu seeder de productos a *DatabaseSeeder.php* para que se lance automáticamente tras las migraciones. Si todo va bien, la aplicación estará lista para responder en <http://localhost/laravel/prueba/public/product>.